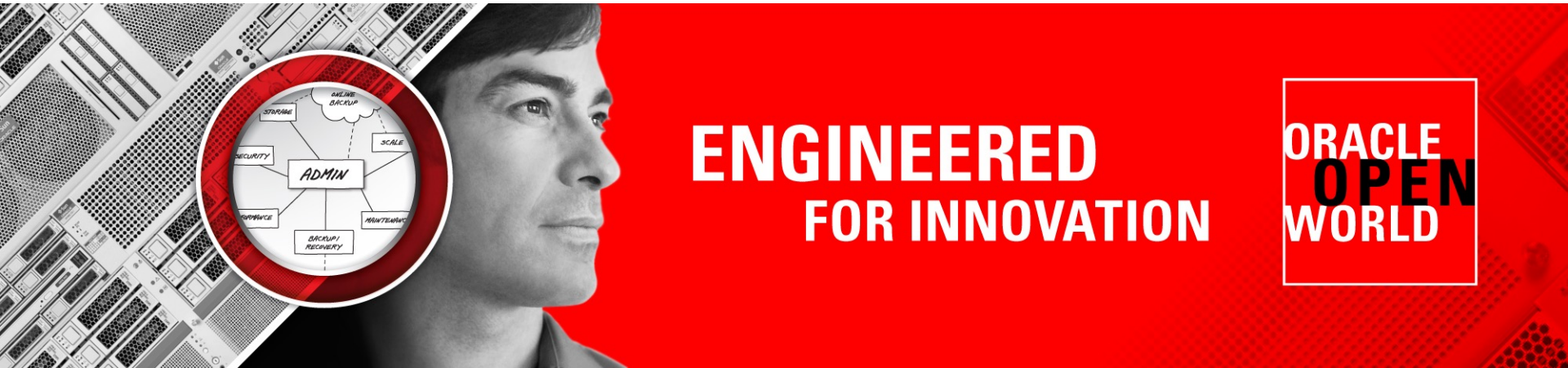




Cours PL/SQL

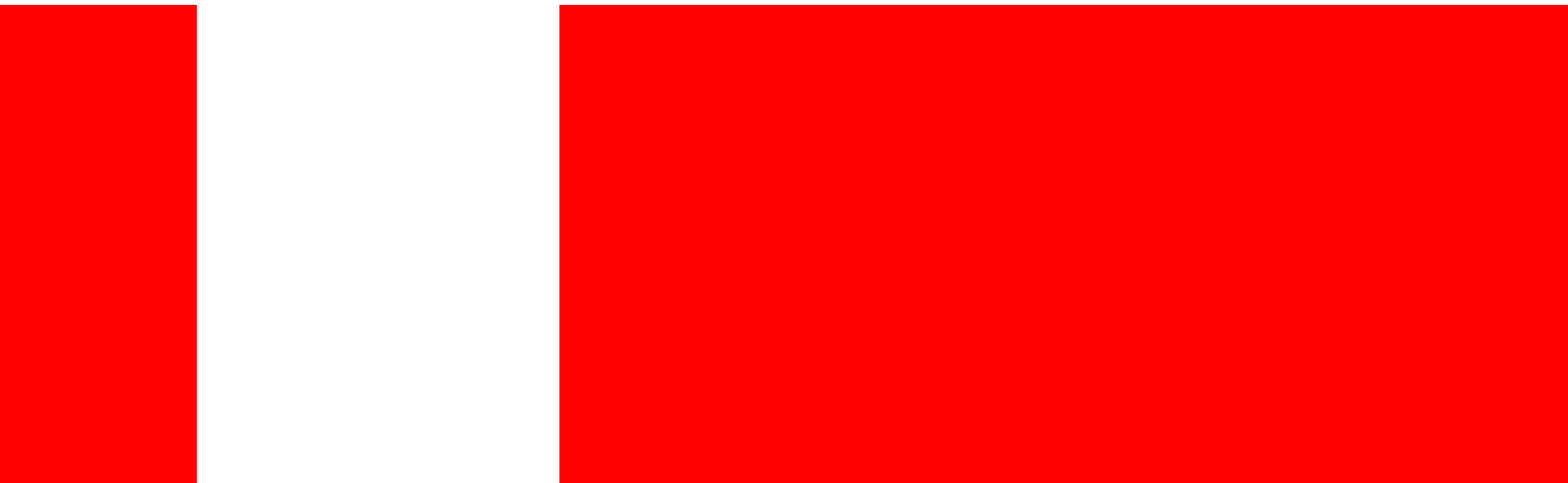
Présenté par: Mme Olfa DRIDI

Cours PL/SQL

CHAPITRE 1: LE LANGAGE SQL

Préparé par: Olfa DRIDI

:

Two large, solid red rectangles are positioned at the top of the slide. The one on the left is a vertical rectangle, and the one on the right is a wider horizontal rectangle.

ORACLE®

**Langage de Définition de Données
&
Langage de Contrôle de Données
&
langage d'interrogation de Données**

SQL

- SQL signifie « Structured Query Language » c'est-à-dire Langage d'interrogation structuré.
- SQL est un langage standardisé, implémenté par tous les SGBDR*, qui permet, indépendamment de la plate- forme technologique et de façon déclarative, de définir le modèle de données, de le contrôler et enfin de le manipuler.
- SQL est un langage complet de gestion de bases de données relationnelles. Il a été conçu par IBM dans les années 70. Il est devenu le langage standard des systèmes de gestion de bases de données (SGBD) relationnelles (SGBDR).
- La norme SQL2 (appelée aussi SQL92) date de 1992.
- SQL3 (appelée aussi SQL99) est la nouvelle norme SQL.
- SQL est un langage ensembliste et non procédural. Les traitements complexes sont parfois difficiles à écrire si on ne peut utiliser des variables et les structures de programmation comme les boucles et les alternatives

SQL

Le langage SQL s'utilise principalement de trois manières :

- Un programme écrit dans un langage de programmation donné utilise l'[interface de programmation](#) du SGBD pour lui transmettre des instructions en langage SQL. Ces programmes utilisent des composants logiciels tels que [ODBC](#) ou [JDBC](#). Cette technique est utilisée par l'[invite de commande](#) qui permet à un [administrateur](#) d'effectuer des opérations sur les bases de données, opérations qu'il décrit en SQL ;
- Technique dite *embedded SQL* : des instructions en langage SQL sont incorporées dans le [code source](#) d'un programme écrit dans un autre langage ;
- Technique des [procédures stockées](#) : des [fonctions](#) écrites dans un langage procédural propre à chaque SGBD (PL/SQL pour Oracle, SQL/PSM pour MySQL ...) sont enregistrées dans la base de données en vue d'être exécutées par le SGBD. Cette technique est aussi utilisée pour les [triggers](#) - procédures déclenchées automatiquement sur modification du contenu de la base de données.

SQL se subdivise en 5 sous- langages:

- **Langage de Définition des Données (LDD** ou en anglais DDL, Data Definition Language) : création, modification et suppression des structures des différents objets de la BD. Les commandes sont: CREATE, ALTER, DROP, RENAME, et TRUNCATE.
- **Langage de Manipulation des Données (LMD** ou en anglais DML, Data Manipulation Language) : ajout, modification et suppression de la BD avec les commandes INSERT, UPDATE, et DELETE.
- **Langage d'Interrogation des données (LID)** : permet de consulter les datas de la BD avec la commande SELECT.
- **Langage de Contrôle des Transactions (LCT)** : les commandes sont COMMIT, ROLLBACK, et SAVEPOINT.
- **Langage de Contrôle de Données (LCD** ou en anglais DCL, Data Control Language) : gestion des protections d'accès: les commandes sont GRANT and REVOKE.

CREATE TABLE

- La commande **CREATE TABLE** permet de créer une table en SQL.
- Une table est une entité qui est contenu dans une base de données pour stocker des données ordonnées dans des colonnes.
- La création d'une table sert à définir les colonnes et le type de données qui seront contenus dans chacun des colonne (entier, chaîne de caractères, date, valeur binaire ...).

CREATE TABLE

```
CREATE TABLE nom_de_la_table (  
    colonne1 type_donnees,  
    colonne2 type_donnees,  
    colonne3 type_donnees,  
    colonne4 type_donnees )
```

- Pour chaque colonne, il est également possible de définir des options telles que (liste non-exhaustive):
 - **NOT NULL** : empêche d'enregistrer une valeur nulle pour une colonne.
 - **DEFAULT valeur**: attribuer une valeur par défaut si aucune donnée n'est indiquée pour cette colonne lors de l'ajout d'une ligne dans la table.
 - **UNIQUE** : Deux n-uplets ne peuvent pas recevoir des valeurs identiques pour cet attribut.
 - **CHECK (condition)** : Vérifie lors de l'insertion de n-uplets que l'attribut réalise la condition condition.
 - **PRIMARY KEY** : indiquer si cette colonne est considérée comme clé primaire pour un index.
 - **FOREIGN KEY REFERENCES table [(colonne)] [ON DELETE CASCADE/ ON DELETE SET NULL]**

Types de données

- Les types de données disponibles en SQL varient d'un SGBD à l'autre, on peut néanmoins citer un certain nombre de types standards que l'on retrouve dans tous les SGBD.
 - INTEGER ou INT, SMALLINT - NUMERIC(X)
 - DECIMAL(X,Y) ou NUMERIC(X,Y)
 - FLOAT(X), REAL
 - CHAR(X)
 - VARCHAR2(X)
 - DATE (AAAA-MM-JJ)
 - DATETIME (AAAA-MM-JJ HH:MM:SS)
 - BOOLEAN pour représenter des booléens,
 - BLOB (pour Binary Long Object) pour représenter des données binaires tels que des documents multimédia (images bitmap, vidéo, etc.)

Exemples

```
SQL> CREATE TABLE nomTable (  
    attribut1 Type1,  
    attribut2 Type2 [NOT NULL | NULL],  
    .....,  
    attributN TypeN ,  
    CONSTRAINT nom_contrainte PRIMARY KEY(attribut1),  
    CONSTRAINT nom_contrainte CHECK(condition),  
    CONSTRAINT NomContrainte FOREIGN KEY (attributN)  
    REFERENCES nomTable2(attribut21) ON DELETE CASCADE) ;
```

Solution de l'exemple2 : Activité (#nomStation, libellé, prix)

```
SQL> CREATE TABLE Activité (  
    nomStation varchar2(15),  
    libellé varchar2(15),  
    prix number(4) NOT NULL,  
    CONSTRAINT activité_PK PRIMARY KEY(nomStation,libellé),  
    CONSTRAINT activité_FK FOREIGN KEY (nomStation) REFERENCES  
    Station(nomStation) ON DELETE CASCADE) ;
```

ALTER TABLE

- Ajout d'une nouvelle colonne à la table avec ses contraintes
- Augmente ou diminuer la largeur d'une colonne existante
- Changer la catégorie d'une colonne, d'obligation à optionnelle ou vice versa (NOT NULL à NULL ou vice versa)
- Spécification d'une valeur par défaut pour une colonne existante
- Changer le type de données d'une une colonne existante
- Spécification d'autres contraintes pour une colonne existante
- Activer ou désactiver une contrainte
- Détruire une contrainte.
- Détruire une colonne
- Renommer une colonne.

ALTER TABLE

La commande ALTER TABLE a les options suivantes:

- ADD
- DROP
- MODIFY
- RENAME
- ENABLE/DISABLE

Exemples

```
CREATE TABLE Employes (  
    NumEmp number(4,0),  
    nom varchar2(15),  
    prenom varchar2(20),  
    ville varchar2(30) );
```

- L'option ADD: permet d'ajouter une colonne ou une contrainte à une table.

```
ALTER TABLE Employes ADD CONSTRAINT emppk PRIMARY  
KEY      (numeemp);
```

→ Permet d'ajouter une contrainte de PK pour la table employes sur l'attribut numeemp.

```
ALTER TABLE Employes ADD (Salaire NUMBER (8,2));
```

→ Permet d'ajouter un attribut Salaire de type NUMBER(8,2) pour la table Employes.

```
ALTER TABLE employes ADD (echelon NUMBER(2,0) NOT  
NULL CHECK (echelon >10))
```

→ Permet d'ajouter une colonne avec ses contraintes

Exemples

L'option **DROP**: permet de supprimer une colonne ou une contrainte à une table.

```
ALTER TABLE Employes DROP PRIMARY KEY;
```

→ C'est possible car une table a une seule clé primaire.

```
ALTER TABLE Employes DROP CONSTRAINT  
cksalaire;
```

```
ALTER TABLE Employes DROP COLUMN nom;
```

Exemples

- L'option **MODIFY**: Cette option permet de modifier le type de données, la valeur par défaut et la contrainte de NOT NULL sur une table déjà existante.

```
ALTER TABLE Employes MODIFY (nom NOT NULL);
```

```
ALTER TABLE Employes MODIFY (nom varchar2(20));
```

```
ALTER TABLE employes MODIFY (ville DEFAULT  
'Montréal');
```

Exemples

- L'option **RENAME**, permet de renommer une contrainte ou une colonne.

```
ALTER TABLE joueurs RENAME CONSTRAINT SYS_C00126194  
TO pkjoueurs;
```

```
ALTER TABLE Employes RENAME COLUMN salaire TO  
SalaireEmp;
```


Exemples

- L'option **DISABLE** ou **ENABLE**, permet de désactiver ou activer une contrainte. (importance de connaître le nom de la contrainte.)

```
ALTER TABLE Employes DISABLE CONSTRAINT cksalaire;
```

```
ALTER TABLE Employes ENABLE CONSTRAINT cksalaire;
```

DROP TABLE

- Permet de supprimer / Détruire une table.

```
DROP TABLE Employees;
```

- A cause de la FOREIGN KEY, les tables référencées ne peuvent être détruites.
- Pour pouvoir supprimer les tables référencées:
 - Soit, vous supprimez les contraintes de FOREIGN KEY d'abord ou les désactiver.
 - Soit supprimez les tables dans un certain ordre. (les tables non référencées en premier)
 - Soit par un CASCADE CONSTRAINT, qui supprime les contraintes de FOREIGN KEY en cascade.

RENAME TABLE

- Permet de renommer une table ou un objet de la base de données.

```
RENAME Employes TO EmployesInfo;
```

Insertion directe de données

```
INSERT INTO <Nom de la relation> (  
    <Liste ordonnée des propriétés à valoriser>  
    VALUES (<Liste ordonnée des valeurs à affecter aux  
    propriétés spécifiées ci-dessus>);
```

Exemples:

- `INSERT INTO livre (titre) VALUES ('L' 'Attrape-cœurs');`
- `INSERT INTO livre (pubdate) VALUES (TO_DATE('20170330', 'YYYYMMDD'));`
- `INSERT INTO Virement (Date, Montant, Objet) VALUES (14-07-1975, 1000, 'Prime de naissance');`

Insertion de données par sélection de valeurs existantes dans la base

```
INSERT INTO <Nom de la relation> (<Liste ordonnée  
des propriétés à valoriser>)  
SELECT ...
```

- Exemple:

```
INSERT INTO Credit (Date, Montant, Objet)  
SELECT Date, Montant, 'Annulation de débit'  
FROM Debit  
WHERE Debit.Date = 25-12-2001;
```

Mise-à-jour de données

UPDATE <Nom de la relation>

SET <Liste d'affectations Propriété = Valeur,
Propriété = Valeur>

WHERE <Condition pour filtrer les tuples à mettre à jour>;

- Exemples:

- **UPDATE** Compte **SET** Monnaie='Euro'
WHERE Monnaie='Franc' ;

- **UPDATE** Compte **SET** Total=Total * 6,55957
WHERE Monnaie='Euro' ;

Suppression de données

```
DELETE FROM <Nom de la relation>  
WHERE <Condition pour filtrer les tuples à  
supprimer>;
```

- Exemples:

- **DELETE FROM** Factures;

- **DELETE FROM** Factures **WHERE** Auteur = 'XXX';

Suppression d'objets

DROP <type objet> <nom objet>;

- **Exemples:**

- DROP TABLE Personne;
- DROP VIEW Employe;



Langage d'interrogation de Données (LID)

- La requête de sélection ou question est la base de la recherche de données en SQL.
- La sélection est la composition d'un produit cartésien, d'une restriction et d'une projection (ou encore la composition d'une jointure et d'une projection).

```
SELECT liste d'attributs projetés  
FROM liste de relations du produit cartésien  
WHERE condition de la restriction;
```

SELECT

- Exemples:
 - **SELECT** Nom, Prenom **FROM** Personne **WHERE** Age>18;
 - **SELECT** Personne.Nom, Personne.Prenom, Vol.Depart
FROM Personne, Vol
WHERE Personne.Vol=Vol.Numero;
- Les opérateurs de comparaison: $P=C$, $P \neq C$, $P < C$, $P > C$, $P \leq C$, $P \geq C$, P BETWEEN C1 AND C2, P IN (C1, C2, ...) , P IS NULL, P LIKE 'chaîne'.
 - L'opérateur LIKE permet d'effectuer une comparaison partielle. Il est surtout employé avec les colonnes contenant des données de type alpha. Il utilise % et _
 - % désigne 0 ou plusieurs caractères quelconques
 - _ désigne 1 et 1 seul caractère
- Les opérateurs logiques: and, or et not.

SELECT

Donner le nom des directeurs	SQL>SELECT nom FROM EMPLOYES WHERE travail = 'Directeur';
Donner le nom des employés dont le salaire est compris entre 1 et 1500 euros	SQL> SELECT nom FROM EMPLOYES WHERE salaire BETWEEN 1 AND 1500;
Donner toutes les caractéristiques des employés dont le nom commence par un D	SQL>SELECT * FROM EMPLOYES WHERE nom LIKE 'D%';
Donner les caractéristiques des employés du département 10, 30 ou 40	SELECT * FROM EMPLOYES WHERE depart IN (10, 30, 40);
Donner les caractéristiques des employés qui n'ont pas de commission	SELECT * FROM EMPLOYES WHERE commission IS NULL;

SELECT DISTINCT

- L'opérateur **SELECT** n'élimine pas les doublons (i.e. les tuples identiques dans la relation résultat) par défaut.
- Il faut pour cela utiliser l'opérateur **SELECT DISTINCT**.

Fonctions d'agrégation

- Une fonction d'agrégation (ou fonction de regroupement) s'applique aux valeurs du sous-ensemble d'un agrégat e relation avec pour résultat la production d'une valeur atomique unique (entier, chaîne, date, etc).
- **COUNT(e)** : Le nombre d'occurrences de e dans le groupe.
 - Les n-uplets pour lesquels e vaut NULL ne sont pas comptés.
 - * peut remplacer e. Compte alors le nombre de n-uplets du groupe.
- **MAX(e)** : La valeur maximale de e pour le groupe.
- **MIN(e)** : La valeur minimale de e pour le groupe.
- **SUM(e)** : La somme des valeurs de e pour le groupe.
- **AVG(e)** : La moyenne de l'évaluation de e sur le groupe.
- **STDDEV(e)** : L'écart-type de e pour le groupe.
- **VARIANCE(e)** : La variance de e pour le groupe.

Les fonctions SQL

- Traitement numérique: Il existe également de nombreuses fonctions mathématiques pour effectuer des calculs ou des statistiques concernant les données contenues dans une base de données.

ABS(n)	Valeur absolue $ABS(-32.5)=32.5$
CEIL(n)	Plus petit entier relatif égal ou supérieur $CEIL(31.5)=32$ $CEIL(-31.5)=-31$
FLOOR(n)	Plus grand entier relatif inférieur ou égal $FLOOR(31.5)=31$ $FLOOR(-31.5)=-32$
MOD(m,n)	Reste de la division de m par n exp: $MOD(35,4)=3$
POWER(m,n)	m puissance n exp: $POWER(4,3)=84$
SIGN(n)	Indique le signe de n $SIGN(0)=0$ $SIGN(-5)=-1$ $SIGN(5)=1$
SQRT(n)	Racine carrée de n $SQRT(9)=3$ $SQRT(-9)=NULL$
ROUND(n)	Arrondi de n à 100 (Partie entière) $ROUND(15.3)=15$ $ROUND(15.5)=16$
TRUNC(n)	n tronqué à 100 (Partie entière) $TRUNC(15.3)=15$

Les fonctions SQL

- Traitement de chaîne

CONCAT	concaténation : équivalent du
INITCAP	initiales en lettres capitales
LPAD	complément ou troncature à n position à gauche
LTRIM / RTRIM	suppression en tête/queue d'une chaîne
REPLACE	remplacement
RPAD	complément ou troncature à n position à droite

- Exemples:

```
SELECT LPAD('azerty', 4, 'x'); -- retourne 'azer'
SELECT LPAD('azerty', 6, 'x'); -- retourne 'azerty'
SELECT LPAD('azerty', 10, 'x'); -- retourne 'xxxxazerty'
SELECT LPAD('azerty', 9, 'xy'); -- retourne 'xyxazerty'
SELECT LPAD('123', 6, '0'); -- retourne '000123'
```


Les fonctions SQL

- Traitement de date

ADD_MONTH, ADD_DAY, ADD_YEAR ...	ajoute des mois, des jours des années à une date
LAST_DAY	renvoie le n° du dernier jour d'un mois d'une date
MONTH_BETWEEN	nombre de mois entre deux dates
NEXT_DAY	date du prochain jour d'un nom donné
TO_CHAR	date sous forme littérale
DATE_DIFF	différence entre deux dates (en nombre de jours)

- Exemples:

```
SELECT sysdate FROM dual;
```

- Opérations mathématiques sur les dates :

```
SELECT sysdate + 10 FROM dual;
```

Les fonctions SQL

- Traitement des dates

- La gestion des dates peut poser des problèmes selon les formats paramétrés sur le serveur Oracle (en général par défaut le format est DD-MON-YY). La solution la plus rigoureuse consiste à utiliser la fonction de conversion TO_DATE.
- TO_DATE convertit une chaîne de caractères à une date selon un format spécifié.

```
TO_DATE( string1 [, format])
```

YYYY: 4-digit year	D: Day of week (1-7).	YEAR: Year, spelled out
MM: Month (01-12; JAN = 01).	DAY: Name of day.	HH/ HH12: Hour of day (1-12).
MON: Abbreviated name of month.	DD: Day of month (1-31).	HH24: Hour of day (0-23).
MONTH: Name of month, padded with blanks to length of 9 characters.	DDD: Day of year (1-366).	MI: Minute (0-59) SS: Second (0-59).

Les fonctions SQL

- Exemples:

SQL> SELECT **TO_DATE**('10-12-11','**MM-DD-YY**') FROM dual; • 12/10/2011

SQL> SELECT **TO_DATE**('10-12-11','**DD-MM-YY**') FROM dual; • 10/12/2011

SQL> SELECT **TO_DATE**('12-31-2011 07:15','**MM-DD-YYYY HH:MI**') FROM dual; • 31/12/2011 07:15:00

SQL> SELECT **TO_DATE**('2011,091' , '**YYYY,DDD**') FROM dual; • 01/04/2011

SQL> SELECT **TO_DATE**('Avril 02, 2011, 11:00','**Month dd, YYYY, HH:MI**') FROM dual; • 02/04/2011 11:00:00



- Voir l'URL suivante pour plus d'informations:

<https://sql.sh/fonctions/date-heure>

La table DUAL

- La **table DUAL** est une table spéciale d'une seule colonne et d'une seule ligne présente par défaut dans toutes les installations du [SGBD Oracle](#). Elle est utilisée généralement pour sélectionner les pseudo-colonnes telles que SYSDATE ou USER.
- La table contient une seule colonne de type [VARCHAR2](#)(1) appelée DUMMY qui a pour unique valeur 'X'.
- Exemple:

```
SELECT 1+1 FROM DUAL; SELECT 1 FROM DUAL;
```

```
SELECT USER FROM DUAL;
```

```
SELECT SYSDATE FROM DUAL;
```

```
SELECT * FROM DUAL
```

Les fonctions SQL

- Conversion
 - Conversion implicite
 - Conversion explicite : TO_DATE, TO_NUMBER, TO_CHAR
- Générales
 - NVL (par exemple NVL(X,0) renvoie 0 si X vaut Null)
 - CASE WHEN condition1 THEN valeur1 WHEN condition2 THEN valeur2 ELSE valeur3 END
 - Imbrication de fonctions : F3(F2(F1(col,arg1),arg2),arg3)
- Extraction de chaîne
 - La fonction substr(X, A, B) renvoie les B caractères à partir du caractère A dans la chaîne X.

Afficher un message selon une condition

- Dans le langage SQL, la commande “CASE ... WHEN ...” permet d'utiliser des conditions de type “si / sinon” (if / else) similaire à un langage de programmation pour retourner un résultat disponible entre plusieurs possibilités. Le CASE peut être utilisé dans n'importe quelle instruction ou clause, telle que SELECT, UPDATE, DELETE, WHERE, ORDER BY ou HAVING.

```
SELECT id, nom, marge_pourcentage, prix_unitaire, quantite,  
       CASE  
         WHEN marge_pourcentage=1 THEN 'Prix ordinaire'  
         WHEN marge_pourcentage>1 THEN 'Prix supérieur à la normale'  
         ELSE 'Prix inférieur à la normale'  
       END  
FROM `achat`
```



id	nom	surcharge	prix_unitaire	quantite	CASE
1	Produit A	1.3	6	3	Prix supérieur à la normale
2	Produit B	1.5	8	2	Prix supérieur à la normale
3	Produit C	0.75	7	4	Prix inférieur à la normale
4	Produit D	1	15	2	Prix ordinaire

GROUP BY

```
SELECT Societe.Nom, AVG(Personne.Age)
FROM Personne, Societe
WHERE Personne.NomSoc = Societe.Nom
GROUP BY Societe.Nom
```

Nom	Age		Nom	AVG(Age)
Oracle	45		Oracle	40
Oracle	35		IBM	25
IBM	20			
IBM	25			
IBM	30			



Regroupement avec un attribut et une fonction

HAVING

- La clause HAVING permet d'effectuer une seconde restriction après l'opération d'agrégation.
- Il est possible d'effectuer **une restriction sur le résultat provenant du calcul de la fonction de groupe**. Dans ce cas, la restriction n'est plus derrière le WHERE mais derrière la clause **HAVING** ; ceci pour indiquer à SQL que la restriction porte sur une donnée calculée et non pas un champ défini dans une table.

```
SELECT Societe.Nom, AVG(Personne.Age)
FROM Personne, Societe
WHERE Personne.NomSoc = Societe.Nom
GROUP BY Societe.Nom
HAVING COUNT(Personne.NumSS) > 2;
```


ORDER BY

- La clause ORDER BY permet de trier les n-uplets du résultat et sa syntaxe est la suivante :

```
ORDER BY expression [ ASC | DESC ] [, ...]
```

- L'ordre ascendant (ASC) ou descendant (DESC) peut être spécifié, sachant qu'il est **ascendant par défaut**.



```
SELECT Num Dept, COUNT(DISTINCT Fonction)
NbFonctions
FROM Employe
WHERE Salaire > 15000
GROUP BY Num Dept
HAVING COUNT(*) > 2;
```

Opérateur de concaténation

- L'opérateur || permet de concaténer des champs de type caractères.

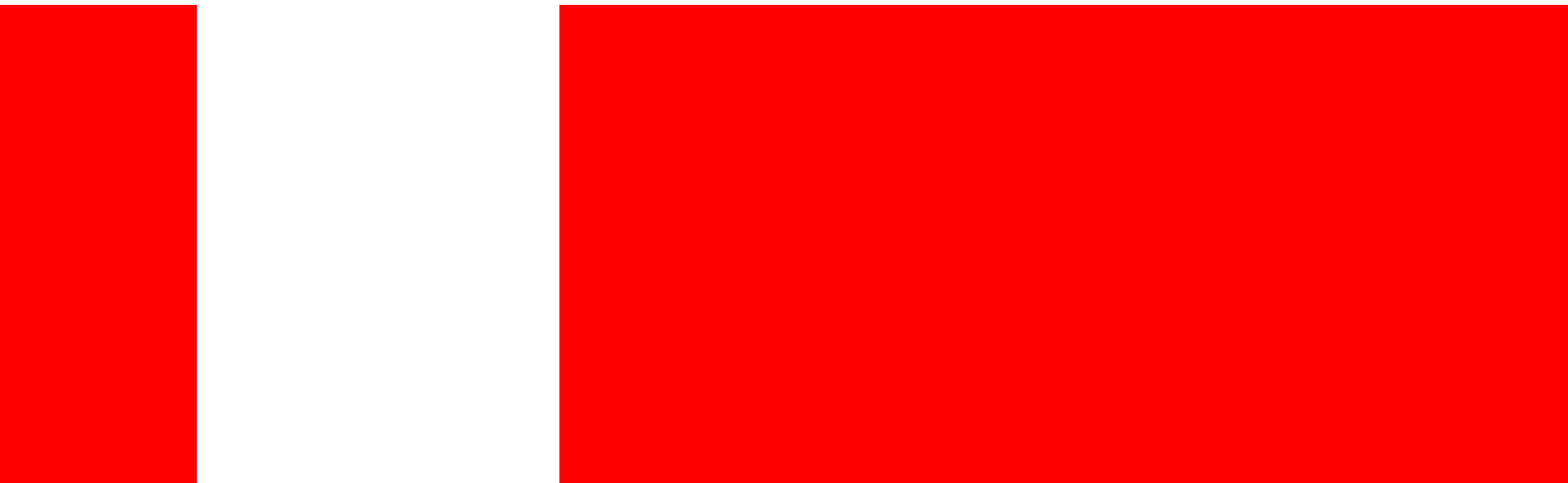
```
SELECT TIT_CODE || ' ' || CLI_PRENOM ||  
' ' || CLI_NOM as NOM  
FROM T_CLIENT;
```

SELECT

```
SELECT [ ALL | DISTINCT ] { * | expression [ AS nom_affiché ] }  
[, ...]  
FROM nom_table [ [ AS ] alias ] [, ...]  
[ WHERE prédicat ]  
[ GROUP BY expression [, ...] ]  
[ HAVING condition [, ...] ]  
[ {UNION | INTERSECT | EXCEPT [ALL]} requête ]  
[ ORDER BY expression [ ASC | DESC ] [, ...] ]
```

Ordre de résolution des requête SQL

- L'ordre de résolution standard d'une requête SQL est :
 - FROM
 - WHERE
 - GROUP BY
 - HAVING
 - SELECT
 - ORDER BY



ORACLE®

Questions ???